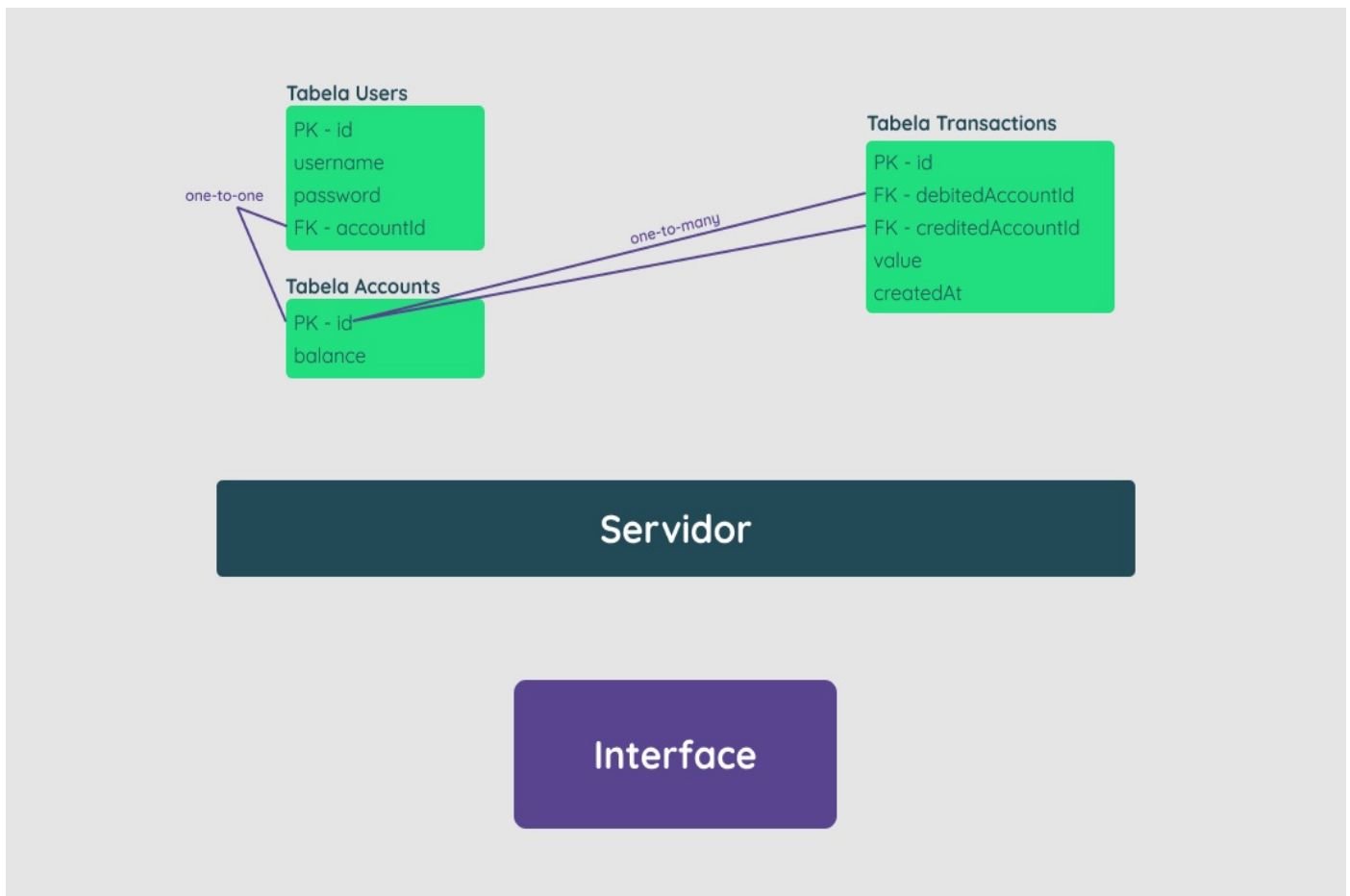


Desafio:

Estruturar uma aplicação web *fullstack*, *dockerizada*, cujo objetivo seja possibilitar que usuários do sistema consigam realizar transferências internas entre si.

- **Backend**
 - **Stack Base**
 - Um servidor em Node.js utilizando NestJS escrito em Typescript;
 - Usando TypeORM
 - Um banco de dados PostgreSQL.
 - **Arquitetura** (Veja o diagrama abaixo p/ entender melhor)
 - **Tabela Users:**
 - id —> *PK*
 - username (o @ do usuário)
 - password (*hashada*)
 - accountId —> *FK Accounts[id]*
 - **Tabela Accounts:**
 - id —> *PK*
 - balance
 - **Tabela Transactions:**
 - id —> *PK*
 - debitedAccountId —> *FK Accounts[id]*
 - creditedAccountId —> *FK Accounts[id]*
 - value
 - createdAt
 - **As seguintes regras de negócio devem ser levadas em consideração durante o processo de estruturação dos endpoints:**
 - Qualquer pessoa deverá poder fazer parte da plataforma. Para isso, basta realizar o cadastro informando *username* e *password*.
 - Deve-se garantir que cada *username* seja único e composto por, pelo menos, 3 caracteres.
 - Deve-se garantir que a *password* seja composta por pelo menos 8 caracteres, um número e uma letra maiúscula. Lembre-se que ela deverá ser *hashada* ao ser armazenada no banco.
 - Durante o processo de cadastro de um novo usuário, sua respectiva conta deverá ser criada automaticamente na tabela **Accounts** com um *balance* de R\$ 100,00. É importante ressaltar que caso ocorra algum problema e o usuário não seja criado, a tabela **Accounts** não deverá ser afetada.
 - Todo usuário deverá conseguir logar na aplicação informando *username* e *password*. Caso o login seja bem-sucedido, um token JWT (com 24h de validade) deverá ser fornecido.
 - Todo usuário logado (ou seja, que apresente um token válido) deverá ser capaz de visualizar seu próprio *balance* atual. Um usuário A não pode visualizar o *balance* de um usuário B, por exemplo.
 - Todo usuário logado (ou seja, que apresente um token válido) deverá ser capaz de realizar um *cash-out* informando o *username* do usuário que sofrerá o *cash-in*, caso apresente *balance* suficiente para isso. Atente-se ao fato de que um usuário não deverá ter a possibilidade de realizar uma transferência para si mesmo.
 - Toda nova transação bem-sucedida deverá ser registrada na tabela **Transactions**. Em casos de falhas transacionais, a tabela **Transactions** não deverá ser afetada.
 - Todo usuário logado (ou seja, que apresente um token válido) deverá ser capaz de visualizar as transações financeiras (*cash-out* e *cash-in*) que participou. Caso o usuário não tenha participado de uma determinada transação, ele nunca poderá ter acesso à ela.
 - Todo usuário logado (ou seja, que apresente um token válido) deverá ser capaz de filtrar as transações financeiras que participou por:
 - Data de realização da transação e/ou
 - Transações de *cash-out*;
 - Transações de *cash-in*.
- **Frontend**
 - **Stack Base**
 - VueJS utilizando Typescript;
 - CSS3 ou uma biblioteca de estilização de sua preferência;
 - **As seguintes regras de negócio devem ser levadas em consideração durante a estruturação da interface visual:**
 - Página para realizar o cadastro na Plataforma informando *username* e *password*.
 - Página para realizar o login informando *username* e *password*.
 - Com o usuário logado, a página principal deve apresentar:
 - *balance* atual do usuário;
 - Seção voltada à realização de transferências para outros usuários da plataforma a partir do *username* de quem sofrerá o *cash-in*;
 - Tabela com os detalhes de todas as transações que o usuário participou;
 - Mecanismo para filtrar a tabela por data de transação e/ou transações do tipo *cash-in/cash-out*;
 - Botão para realizar o *log-out*.
- **Diagrama**



- Observações Finais
 - boas práticas e padrões de desenvolvimento são apreciados